

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
«Конструювання програмного забезпечення»

Лабораторна робота № 2
Варіант 1

Виконав:	Приймаченко Артем Андрійович	Перевірів:	Шабатін Павло Євгенійович
Група	ІПЗ-34-10	Дата перевірки	
Форма навчання	денна	Оцінка	
Спеціальність	121		
2026			

Варіант завдання: Симулятор системи залежностей (Dependency resolver).

Хід роботи:

Почнемо з точки входу програми:

```
0 references
private static void Main(string[] args)
{
    try
    {
        string filePath = args[0];

        var lines = new FileReader().ReadLines(filePath);
        DependencyGraph graph = new GraphBuilder().BuildGraph(lines);
        DependencyResolver resolver = new DependencyResolver(graph);
        List<Package> order = resolver.Resolve();

        Console.WriteLine("Package installation order:\n");

        for (int i = 1; i < order.Count; i++)
        {
            Console.WriteLine(i + ". " + order[i].name + "-" + order[i].version);
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine("An error occurred: " + ex.Message);
    }
}
```

Шлях до файлу повинен надаватись користувачем при запуску програми, наприклад:

```
C:\Users\sarew>dotnet run file.txt
```

Спочатку передамо файл єдиному методу класу FileReader для його зчитування. Метод повинен повернути послідовність рядків (де кожен елемент типу string – окремий рядок:

```

1 reference
public class FileReader
{
    /// <summary>
    /// Reads all lines from the specified file.
    /// </summary>
    /// <param name="filePath">The path to the file to read.
    /// <returns>An <see cref="IEnumerable{String}"/> conta
    1 reference
    public IEnumerable<string> ReadLines(string filePath)
    {
        return File.ReadLines(filePath);
    }
}

```

Після цього йде черга класу GraphBuilder. Метод BuildGraph приймає створену раніше послідовність рядків, і повинен пройтись по кожному. Якщо знайдено рядок з пакетом (але не тим, якого потребує інший пакет), то даний пакет додається в граф. Після цього йде пошук всіх пакетів, від якого поточний залежить. По кожному з них проходимося, додаючи як пакети, так і зв'язки між ними. Після завершення обходу всіх рядків метод поверне граф:

```

public DependencyGraph BuildGraph(IEnumerable<string> lines)
{
    DependencyGraph graph = new DependencyGraph();

    string packagePattern = @"package:";
    string requiresPattern = @"requires:";

    var lineList = lines.ToList();

    int i = 0;

    for (; i < lineList.Count; i++)
    {
        string line = lineList[i];

        if (!line.Contains(packagePattern))
        {
            continue;
        }

        this.FindPackage(line, packagePattern, out Package package);
        graph.AddPackage(package);

        int j = i + 1;

        while (j < lineList.Count && lineList[j].Contains(requiresPattern))
        {
            this.FindPackage(lineList[j], requiresPattern, out Package dependencyPackage);
            if (dependencyPackage != null)
            {
                graph.AddPackage(dependencyPackage);
                graph.AddDependency(package, dependencyPackage);
            }

            j++;
        }

        i = j - 1;
    }

    return graph;
}

```

Метод, що шукає назву та версію пакету в поточному рядку:

```

2 references
private void FindPackage(string line, string pattern, out Package? package)
{
    string result = Regex.Replace(line, pattern, string.Empty, RegexOptions.IgnoreCase);
    if (string.IsNullOrEmpty(result.Trim()))
    {
        package = null;
        return;
    }

    string[] parts = result.Split(',', StringSplitOptions.TrimEntries);
    package = new Package(parts[0], parts[1]);
}

```

Тут видаляється початкова частина рядка, і залишається тільки назва та версія пакету, що розділені комою.

Пакети представлено у вигляді запису. Хоча й можна було б створити примітивний клас, але далі нам важливо буде порівнювати об'єкти саме по їхніх значеннях, а не по посиланнях. У випадку використання класу нам потрібно було б як мінімум перевизначити метод Equals, щоб порівняння по значенню було доступним. Але записи надають це в базовому варіанті, тому зручно використовувати саме їх.

```
38 references  
public record Package(string name, string version);
```

Перейдемо до класу DependencyGraph, що відповідальний за керування графом залежностей.

Він зберігатиме граф у вигляді словника:

```
11 references  
public class DependencyGraph  
{  
    private Dictionary<Package, List<Package>> dependencies;  
  
    /// <summary>  
    /// Initializes a new instance of the <see cref="DependencyGraph"/> class.  
    /// </summary>  
    1 reference  
    public DependencyGraph()  
    {  
        this.dependencies = new Dictionary<Package, List<Package>>();  
    }  
}
```

Метод для додавання пакету. Тут просто перевіряємо, чи не існує такого пакету на даний момент. Якщо ні – додаємо його в граф:

```
2 references  
public void AddPackage(Package package)  
{  
    if (!this.dependencies.ContainsKey(package))  
    {  
        this.dependencies.Add(package, new List<Package>());  
    }  
}
```

Для додавання залежності метод повинен отримати два пакети (між якими цю залежність потрібно додати), і додає її:

```
1 reference  
public void AddDependency(Package from, Package to)  
{  
    this.dependencies[from].Add(to);  
}
```

Для отримання залежностей конкретного пакету відповідальний метод `GetDependencies`:

```
1 reference
public List<Package> GetDependencies(Package package)
{
    if (this.dependencies.ContainsKey(package))
    {
        return this.dependencies[package];
    }
    else
    {
        return new List<Package>();
    }
}
```

Тут перевіряємо, чи переданий пакет існує. Якщо так – повертаємо всі його залежності. Інакше повернемо порожній список. Цей метод, як і ще один нижче, будуть використовуватись при виявленні залежностей у графі.

А цей метод просто повертає всі пакети:

```
1 reference
public IEnumerable<Package> GetAllPackages()
{
    return this.dependencies.Keys;
}
```

Тепер можемо перейти до класу, відповідального за знаходження залежностей.

Ми створимо структуру `HashSet` (без дублікатів), щоб зберігати відвідані пакети, стек (для тимчасового зберігання порядку), отримуємо всі пакети, а також в `HashSet` зберігатимемо шлях (що складається із пакетів) на певному етапі (це потрібно для виявлення циклічних залежностей).

```
1 reference
public List<Package> Resolve()
{
    HashSet<Package> visited = new HashSet<Package>();
    Stack<Package> stack = new Stack<Package>();
    IEnumerable<Package> packages = this.graph.GetAllPackages();
    HashSet<Package> currentPath = new HashSet<Package>();

    foreach (Package package in packages)
    {
        if (!visited.Contains(package))
        {
            this.DepthFirstSearch(package, visited, stack, currentPath);
        }
    }
}
```

Пройдемо по кожному пакету. Якщо пакет ще не був відвіданий, викликаємо метод для пошуку залежностей поточного пакету:

```
2 references
private void DepthFirstSearch(
    Package currentPackage,
    HashSet<Package> visited,
    Stack<Package> stack,
    HashSet<Package> currentPath)
{
    visited.Add(currentPackage);
    currentPath.Add(currentPackage);

    List<Package> dependencyPackages = this.graph.GetDependencies(currentPackage);

    foreach (Package dependency in dependencyPackages)
    {
        if (currentPath.Contains(dependency))
        {
            throw new Exception("Dependency cycle detected");
        }

        if (!visited.Contains(dependency))
        {
            this.DepthFirstSearch(dependency, visited, stack, currentPath);
        }
    }

    currentPath.Remove(currentPackage);
    stack.Push(currentPackage);
}
```

Додаємо поточний пакет в список відвіданих, а також в колекцію із поточним шляхом. Також отримуємо всі його залежності (у вигляді списку). Проходимо по кожній залежності із цього списку. Перевіряємо, чи не є якась із його залежностей в колекції поточного шляху. Якщо таке трапляється, то граф містить циклічну залежність, і припиняється робота програми. Якщо такого не виявлено, перевіряємо, чи не містить колекція відвіданих вузлів поточний пакет. Якщо містить – пропускаємо його, якщо не містить – викликаємо цей самий метод, але передаючи вже поточний пакет. Коли пройдемо по всіх пакетах, очищуємо поточний шлях, а також додаємо пакет (який ми передавали в цей метод) у стек.

Коли все це здійснимо для всіх пакетів, додамо всі елементи стеку в список, і відсортуємо його елементи таким чином, щоб вони йшли в зворотньому порядку. Таким чином буде отримано коректний порядок пакетів для встановлення.

```

public List<Package> Resolve()
{
    HashSet<Package> visited = new HashSet<Package>();
    Stack<Package> stack = new Stack<Package>();
    IEnumerable<Package> packages = this.graph.GetAllPackages();
    HashSet<Package> currentPath = new HashSet<Package>();

    foreach (Package package in packages)
    {
        if (!visited.Contains(package))
        {
            this.DepthFirstSearch(package, visited, stack, currentPath);
        }
    }

    List<Package> packagesSorted = new List<Package>();
    while (stack.Count != 0)
    {
        packagesSorted.Add(stack.Pop());
    }

    packagesSorted.Reverse();

    return packagesSorted;
}

```

В кінці виведемо порядок пакетів у класі Program:

```

Console.WriteLine("Package installation order:\n");

for (int i = 1; i < order.Count; i++)
{
    Console.WriteLine(i + ". " + order[i].name + "-" + order[i].version);
}

```

Висновки.

У цій роботі було створено основну частину програми. Спочатку було вирішено відкоригувати структуру проекту, виокремивши зчитування файлу та побудову графу в два різні класи, таким чином уникнувши порушення принципу єдиної відповідальності. На цьому етапі програма може здійснювати всю потрібну базову логіку: зчитування файлу, побудова графу, виявлення залежностей, а також виведення результату, у випадку якщо на вхід подано файл з передбаченим змістом.